

The Cosmic Gate — Super Notebook

Black Hole Information, Holographic Retirement, the SHA Die, and the Full Nexus Execution Stack

Driven by Dean W. Kulik
Drafted in collaboration with ChatGPT
Date: April 2, 2026

Scope

This is a **super notebook**: a full paper companion, not a thin computational appendix.

It integrates four layers into one executable object:

1. **Die core** — the SHA-256 register machine, ground witness, nilpotent backbone, seam differential, support laws, orbit closures.
2. **Address layer** — the BBP pointer engine as $O(1)$ -style block access into the π manifold.
3. **Cosmic benchmark layer** — Schwarzschild radius, horizon area, Bekenstein–Hawking entropy, and solar-mass scale checks.
4. **Macro-lift layer** — the black-hole information paper's holographic retirement, Glass Key analogy, and controller logic, kept structurally separate from the hard computational core.

The rule of this notebook is strict:

- when something can be **directly executed and checked**, it is,
- when something is a **structural lift or isomorphism**, it is labeled as such,
- and when an unresolved seam appears, it is treated as **structured absence**, not as silence.

The spine is:

$$\boxed{\text{shape} \rightarrow \text{constraint} \rightarrow \text{transition} \rightarrow \text{retention} \rightarrow \text{projection}}$$

This notebook is self-contained.

Part I — Core environment and primitive operators

We begin with the hard die layer. This is the same executable substrate that all later lifts depend on.

The working state is

$$\begin{matrix} x_r \\ \vdots \\ a_r \\ b_r \\ c_r \\ d_r \\ e_r \\ f_r \\ g_r \\ h_r \end{matrix} \in (\mathbb{Z}/2^{32}\mathbb{Z})^8$$

with round functions

$$T1_r = h_r + \Sigma_1(e_r) + \text{Ch}(e_r, f_r, g_r) + K_r + W_r$$

$$T2_r = \Sigma_0(a_r) + \text{Maj}(a_r, b_r, c_r)$$

$$a_{r+1} = T1_r + T2_r, \quad e_{r+1} = d_r + T1_r$$

and pure transport on the remaining six lanes.

```
In [13]: import math
import random
import statistics
import hashlib
from typing import List, Tuple, Dict

import numpy as np
import mpmath as mp

MASK32 = 0xFFFFFFFF
mp.mp.dps = 120

def rotr(x: int, n: int) -> int:
    x &= MASK32
    return ((x >> n) | ((x << (32 - n)) & MASK32)) & MASK32

def Sigma0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)
```

```

def Sigma1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def Ch(e: int, f: int, g: int) -> int:
    return ((e & f) ^ ((~e) & g)) & MASK32

def Maj(a: int, b: int, c: int) -> int:
    return (a & b) ^ (a & c) ^ (b & c)

H0 = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]

K = [
    0x428A2F98, 0x71374491, 0xB5C0FBCF, 0xE9B5DBA5, 0x3956C25B, 0x59F111F1, 0x923F82A4, 0xAB1C5ED5,
    0xD807AA98, 0x12835B01, 0x243185BE, 0x550C7DC3, 0x72BE5D74, 0x80DEB1FE, 0x9BDC06A7, 0xC19BF174,
    0xE49B69C1, 0xEFBE4786, 0x0FC19DC6, 0x240CA1CC, 0x2DE92C6F, 0x4A7484AA, 0x5CB0A9DC, 0x76F988DA,
    0x983E5152, 0xA831C66D, 0xB00327C8, 0xBF597FC7, 0xC6E00BF3, 0xD5A79147, 0x06CA6351, 0x14292967,
    0x27B70A85, 0x2E1B2138, 0x4D2C6DFC, 0x53380D13, 0x650A7354, 0x766A0ABB, 0x81C2C92E, 0x92722C85,
    0xA2BFE8A1, 0xA81A664B, 0xC24B8B70, 0xC76C51A3, 0xD192E819, 0xD6990624, 0xF40E3585, 0x106AA070,
    0x19A4C116, 0x1E376C08, 0x2748774C, 0x34B0BCB5, 0x391C0CB3, 0x4ED8AA4A, 0x5B9CCA4F, 0x682E6FF3,
    0x748F82EE, 0x78A5636F, 0x84C87814, 0x8CC70208, 0x90BEFFFA, 0xA4506CEB, 0xBEF9A3F7, 0xC67178F2,
]

def round_terms(state: List[int], W_r: int, r: int) -> Tuple[int, int]:
    a, b, c, d, e, f, g, h = state
    t1 = (h + Sigma1(e) + Ch(e, f, g) + K[r] + W_r) & MASK32
    t2 = (Sigma0(a) + Maj(a, b, c)) & MASK32
    return t1, t2

def round_step(state: List[int], W_r: int, r: int) -> List[int]:
    a, b, c, d, e, f, g, h = state
    t1, t2 = round_terms(state, W_r, r)
    new_a = (t1 + t2) & MASK32
    new_e = (d + t1) & MASK32
    return [new_a, a, b, c, new_e, e, f, g]

def nop_orbit(rounds: int = 64):
    state = H0.copy()
    orbit = []

```

```

for r in range(rounds):
    t1, t2 = round_terms(state, 0, r)
    orbit.append({
        "r": r,
        "state": state.copy(),
        "t1": t1,
        "t2": t2,
        "diff": (state[0] - state[4]) & MASK32,
    })
    state = round_step(state, 0, r)
return orbit

def BBP_pointer(pos: int, block_size: int = 8) -> int:
    """Read a 32-bit (8-hex) block from the hexadecimal expansion of pi at position pos."""
    pi_frac = mp.pi - 3
    pi_frac *= mp.power(16, pos)
    pi_frac = pi_frac - mp.floor(pi_frac)
    block_int = 0
    for _ in range(block_size):
        pi_frac *= 16
        digit = int(pi_frac)
        block_int = (block_int * 16) + digit
        pi_frac -= digit
    return block_int

def SHA_fold(data) -> str:
    if isinstance(data, int):
        data = str(data).encode("utf-8")
    elif isinstance(data, str):
        data = data.encode("utf-8")
    return hashlib.sha256(data).hexdigest()

def double_sha_glass_key(data) -> str:
    if isinstance(data, int):
        data = str(data).encode("utf-8")
    elif isinstance(data, str):
        data = data.encode("utf-8")
    h1 = hashlib.sha256(data).digest()
    return hashlib.sha256(h1).hexdigest()

```

Part II — Hard die anchors

These are the first exact anchors of the machine:

1. the message-free ground witness $T2_0^{(0)} = 0x08909ae5$
2. the first-step displacement identity $\Delta a_1 = W_0 \sqcup \Delta e_1 = W_0$
3. the exact seam differential $a_{r+1} - e_{r+1} \equiv T2_r - d_r \pmod{2^{32}}$

```
In [14]: t1_0, t2_0 = round_terms(H0, 0, 0)
print(f"T2_0^{(0)} = 0x{t2_0:08x}")
assert t2_0 == 0x08909AE5

for W0 in [0, 1, 0x80000000, 0x12345678, 0xFFFFFFFF]:
    base = round_step(H0.copy(), 0, 0)
    live = round_step(H0.copy(), W0, 0)
    da1 = (live[0] - base[0]) & MASK32
    de1 = (live[4] - base[4]) & MASK32
    assert da1 == (W0 & MASK32)
    assert de1 == (W0 & MASK32)

rng = random.Random(20260402)
for _ in range(8000):
    state = [rng.getrandbits(32) for _ in range(8)]
    W_r = rng.getrandbits(32)
    r = rng.randrange(64)
    t1, t2 = round_terms(state, W_r, r)
    new_state = round_step(state, W_r, r)
    lhs = (new_state[0] - new_state[4]) & MASK32
    rhs = (t2 - state[3]) & MASK32
    assert lhs == rhs

print("Verified:")
print(" • T2_0^{(0)} ground witness")
print(" • exact first-step displacement into a/e")
print(" • seam differential identity over 8000 random round states")
```

$T2_0^{(0)} = 0x08909ae5$

Verified:

- $T2_0^{(0)}$ ground witness
- exact first-step displacement into a/e
- seam differential identity over 8000 random round states

Part III — Nilpotent transport and controllability

The pure transport backbone is the 8×8 shift matrix

$$P = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

which satisfies

$$\chi_P(\lambda) = \lambda^8, \text{quad } P^8 = 0.$$

So the backbone is a finite-memory conveyor. Left alone, it dies in at most eight steps.

The machine survives because it is driven through two seams:

$$x_{r+1} = Px_r + u_a(T1_r + T2_r) + u_e T1_r.$$

The controllability matrix should reach full rank 8.

```
In [15]: P = np.array([
    [0,0,0,0,0,0,0,0],
    [1,0,0,0,0,0,0,0],
    [0,1,0,0,0,0,0,0],
    [0,0,1,0,0,0,0,0],
    [0,0,0,1,0,0,0,0],
    [0,0,0,0,1,0,0,0],
    [0,0,0,0,0,1,0,0],
    [0,0,0,0,0,0,1,0],
    [0,0,0,0,0,0,0,1],
    ], dtype=int)

u_a = np.array([1,0,0,0,0,0,0,0], dtype=int)
u_e = np.array([0,0,0,0,1,0,0,0], dtype=int)

P8 = np.linalg.matrix_power(P, 8)
assert np.all(P8 == 0)
assert np.linalg.matrix_rank(P) == 7

B = np.column_stack([u_a, u_e])
```

```

Ctrb = np.column_stack([np.linalg.matrix_power(P, k) @ B for k in range(8)])
rank_C = np.linalg.matrix_rank(Ctrb)
assert rank_C == 8

rank_growth = []
for k in range(1, 9):
    Ck = np.column_stack([np.linalg.matrix_power(P, j) @ B for j in range(k)])
    rank_growth.append(np.linalg.matrix_rank(Ck))

print("rank(P) =", np.linalg.matrix_rank(P))
print("P^8 == 0 ?", np.all(P8 == 0))
print("rank(controllability) =", rank_C)
print("rank growth =", rank_growth)

```

```

rank(P) = 7
P^8 == 0 ? True
rank(controllability) = 8
rank growth = [np.int64(2), np.int64(4), np.int64(6), np.int64(8), np.int64(8), np.int64(8), np.int64(8), np.int64(8)]

```

Part IV — Support closure, waists, and the wave triad

The die admits two distinct closure layers:

Support layer

- word depth $D_{\{\text{word}\}}=4$
- support-level bit depth $D_{\{\text{bit}\}}^{\{\{\text{support}\}\}}=6$
- local waist $w_0 = 6-4 = 2$

Live orbit layer

- live bit depth $D_{\{\text{bit}\}}^{\{\{\text{live}\}\}}=10$
- orbit waist $w_{\Omega} = 10-4 = 6$

The support-layer wave triad is derived from $D_{\{\text{word}\}}=4$ and $D_{\{\text{bit}\}}^{\{\{\text{support}\}\}}=6$:

$$n^2 = \frac{6}{4} = \frac{3}{2}$$

$$K = \sqrt{(4+6) \cdot 6} = \sqrt{60}, \text{qqquad } W = \sqrt{(4+6) \cdot 4} = \sqrt{40}$$

$$K^2 + W^2 = 100$$

and normalized closure

$$R^2 + G^2 = 1$$

with

$$R = \frac{W}{10}, \text{qqquad } G = \frac{K}{10}.$$

```
In [16]: M = np.array([
    [1,1,1,0,1,1,1,1],
    [1,0,0,0,0,0,0,0],
    [0,1,0,0,0,0,0,0],
    [0,0,1,0,0,0,0,0],
    [0,0,0,1,1,1,1,1],
    [0,0,0,0,1,0,0,0],
    [0,0,0,0,0,1,0,0],
    [0,0,0,0,0,0,1,0],
], dtype=bool)
B_word = np.array([1,0,0,0,1,0,0,0], dtype=bool)

def word_support_sequence(rounds: int = 6):
    sigma = np.zeros(8, dtype=bool)
    seq = []
    for r in range(rounds):
        omega = (r == 0)
        sigma = (M @ sigma.astype(int) > 0) | (B_word if omega else False)
        seq.append(sigma.copy())
    return seq

def R_bool(x: np.ndarray, n: int) -> np.ndarray:
    return np.roll(x, n)

def Sigma0_sup(x: np.ndarray) -> np.ndarray:
    return R_bool(x, 2) | R_bool(x, 13) | R_bool(x, 22)

def Sigma1_sup(x: np.ndarray) -> np.ndarray:
```

```

    return R_bool(x, 6) | R_bool(x, 11) | R_bool(x, 25)

def L32(x: np.ndarray) -> np.ndarray:
    out = np.zeros_like(x, dtype=bool)
    acc = False
    for i in range(len(x)):
        acc = acc or bool(x[i])
        out[i] = acc
    return out

def support_radius_for_bit(j: int, max_rounds: int = 20) -> int:
    state = np.zeros((8, 32), dtype=bool)
    omega = np.zeros(32, dtype=bool)
    omega[j] = True
    for r in range(1, max_rounds + 1):
        a,b,c,d,e,f,g,h = state
        tau1 = h | Sigma1_sup(e) | e | f | g | omega
        tau2 = Sigma0_sup(a) | a | b | c
        new = np.zeros_like(state)
        new[0] = L32(tau1 | tau2)
        new[1] = a
        new[2] = b
        new[3] = c
        new[4] = L32(d | tau1)
        new[5] = e
        new[6] = f
        new[7] = g
        state = new
        omega = np.zeros(32, dtype=bool)
        if state.all():
            return r
    raise RuntimeError("support did not close")

seq = word_support_sequence()
D_word = next(i for i, s in enumerate(seq, start=1) if s.all())
radii = [support_radius_for_bit(j) for j in range(32)]
D_bit_support = max(radii)
w0 = D_bit_support - D_word

assert D_word == 4
assert D_bit_support == 6
assert w0 == 2

```

```

n2 = D_bit_support / D_word
scale = D_word + D_bit_support
Ktri = math.sqrt(scale * D_bit_support)
Wtri = math.sqrt(scale * D_word)

assert math.isclose(n2, 3/2)
assert math.isclose(Ktri**2 + Wtri**2, scale**2)

R = Wtri / scale
G = Ktri / scale
assert math.isclose(R*R + G*G, 1.0)

print("D_word =", D_word)
print("D_bit^(support) =", D_bit_support)
print("w0 =", w0)
print("n^2 =", n2)
print("K =", Ktri, "W =", Wtri)
print("R^2 + G^2 =", R*R + G*G)

```

```

D_word = 4
D_bit^(support) = 6
w0 = 2
n^2 = 1.5
K = 7.745966692414834 W = 6.324555320336759
R^2 + G^2 = 1.0

```

Part V — Seven-level orbit closures

The orbit paper and A-Mark9 layer pin the live-orbit invariants:

$D_{\text{word}}=4, \text{quad } D_{\text{bit}}^{\text{live}}=10, \text{quad } w_{\Omega}=6$

$|K_{\text{lie}}|=26, \text{quad } |K_{\text{ground}}|=36, \text{quad } K_{\text{inflect}}=\{32,57\}$

$\tau=3, \text{quad } A_{\text{max}}=224, \text{quad } \text{crossovers}=41$

The internal closures among these quantities are part of the super-notebook because they are load-bearing for later macro-lifts:

$D_{\text{bit}}^{\text{live}}=D_{\text{word}}+w_{\Omega}$

$||K_{\text{ground}}|-|K_{\text{lie}}||=D_{\text{bit}}^{\text{live}} \quad \square$

$|K_{\text{lie}}|+|K_{\text{ground}}|+|K_{\text{inflect}}|=64 \quad \square$

```
In [17]: D_bit_live = 10
w_orbit = 6
K_lie = 26
K_ground = 36
K_inflect = {32, 57}
tau = 3
A_max = 224
crossovers = 41

assert D_bit_live == D_word + w_orbit
assert K_ground - K_lie == D_bit_live
assert K_lie + K_ground + len(K_inflect) == 64
assert tau == w_orbit // 2
assert tau == D_word - 1
assert A_max == 7 * 32 == 256 - 32
assert 57 == 64 - 7

print("Orbit closures verified:")
print(" D_bit^(live) = D_word + w_orbit =", D_bit_live)
print(" |K_ground| - |K_lie| =", K_ground - K_lie)
print(" partition total =", K_lie + K_ground + len(K_inflect))
print(" tau =", tau)
print(" A_max =", A_max)
print(" crossovers =", crossovers)
```

```
Orbit closures verified:
D_bit^(live) = D_word + w_orbit = 10
|K_ground| - |K_lie| = 10
partition total = 64
tau = 3
A_max = 224
crossovers = 41
```

Part VI — BBP pointer engine

The paper's address layer treats BBP as a coordinate engine:

\$\$ \text{\text{address}} \text{\text{to}} \text{\text{phase-lock}} \text{\text{to}} \text{\text{32-bit block readout}} \$\$

At the computational level, this notebook can verify only the mathematical property that BBP provides direct-access block extraction from the hexadecimal expansion of π .

This is the hard part:

- exact block reads at arbitrary offsets,
- no need to compute prior digits as notebook state,
- and correct reproduction of canonical leading words.

```
In [25]: def BBP_pointer(pos: int, block_size: int = 8) -> int:
          """Read an 8-hex-digit block from the hexadecimal expansion of pi at position pos.

          Precision is set dynamically so deeper offsets do not collapse to zero from insufficient dps.
          """
          needed_dps = max(120, int((pos + block_size + 16) * math.log10(16)) + 50)
          old_dps = mp.mp.dps
          mp.mp.dps = needed_dps
          try:
              pi_frac = mp.frac((mp.pi - 3) * mp.power(16, pos))
              block_int = 0
              for _ in range(block_size):
                  pi_frac *= 16
                  digit = int(pi_frac)
                  block_int = (block_int << 4) | digit
                  pi_frac -= digit
              return block_int
          finally:
              mp.mp.dps = old_dps
```

Part VII — Standard black-hole benchmark

This section is deliberately orthodox.

We compute the standard Schwarzschild and Bekenstein–Hawking quantities for a one-solar-mass black hole:

$$r_s = \frac{2GM}{c^2}$$

$$A = 4\pi r_s^2$$

$$S_{\text{BH}} = \frac{k_B c^3}{4 G \hbar} A$$

$$N_{\text{bits}} = \frac{S_{\text{BH}}}{k_B \ln 2}$$

This is the hard benchmark layer the cosmic lift has to pass.

```
In [19]: # SI constants
G_SI = 6.67430e-11          # m^3 kg^-1 s^-2
c_SI = 299792458.0         # m s^-1
hbar_SI = 1.054571817e-34  # J s
kB_SI = 1.380649e-23       # J K^-1
M_sun = 1.98847e30         # kg

def schwarzschild_radius(M: float) -> float:
    return 2 * G_SI * M / (c_SI ** 2)

def horizon_area(M: float) -> float:
    r_s = schwarzschild_radius(M)
    return 4 * math.pi * r_s ** 2

def bekenstein_hawking_entropy_J_per_K(M: float) -> float:
    A = horizon_area(M)
    return (kB_SI * c_SI**3 * A) / (4 * G_SI * hbar_SI)

def bekenstein_hawking_entropy_bits(M: float) -> float:
    S = bekenstein_hawking_entropy_J_per_K(M)
    return S / (kB_SI * math.log(2))

r_s = schwarzschild_radius(M_sun)
A = horizon_area(M_sun)
S_JK = bekenstein_hawking_entropy_J_per_K(M_sun)
S_bits = bekenstein_hawking_entropy_bits(M_sun)

print(f"Schwarzschild radius (1 Msun): {r_s:.6f} m")
print(f"Horizon area (1 Msun): {A:.6e} m^2")
print(f"Entropy S_BH: {S_JK:.6e} J/K")
print(f"Entropy in bits: {S_bits:.6e} bits")
```

```
# broad sanity windows
assert 2.9e3 < r_s < 3.1e3
assert 1.0e77 < S_bits < 2.0e77

print("\nVerified: standard solar-mass black-hole entropy scale.")
```

```
Schwarzschild radius (1 Msun): 2953.339382 m
Horizon area (1 Msun):      1.096066e+08 m^2
Entropy S_BH:                1.448239e+54 J/K
Entropy in bits:             1.513322e+77 bits
```

Verified: standard solar-mass black-hole entropy scale.

Part VIII — Holographic retirement toy model

The black-hole paper's core lift is that the event horizon should be read as a **retirement surface**, not a destruction void.

This notebook cannot directly prove that physical black holes literally execute a SHA-like fold. What it can do is assemble the exact computational components the paper uses for that lift:

1. finite transport into a terminal surface,
2. address resolution via a precomputed manifold,
3. state retirement via one-fold or double-fold hash anchor,
4. finite storage benchmark from horizon area.

That gives the structural stack:

$\text{\$ \$ \text{\{infall\}} \text{\to} \text{\{gate\}} \text{\to} \text{\{transport\}} \text{\to} \text{\{address\}} \text{\to} \text{\{retirement\}} \text{\$ \$}}$

The next cell demonstrates the abstract retirement pipeline with BBP-derived address blocks and SHA / double-SHA anchors.

```
In [20]: toy_addresses = [0, 8, 16, 32]
toy_records = []
for addr in toy_addresses:
    word = BBP_pointer(addr)
    toy_records.append({
        "addr": addr,
        "word_hex": f"0x{word:08x}",
```

```

    "sha": SHA_fold(word),
    "double_sha": double_sha_glass_key(word),
})

```

toy_records

```

Out[20]: [{ 'addr': 0,
  'word_hex': '0x243f6a88',
  'sha': 'e8a8dd5d0e26a2e95028b6fe08db86bf802d3b7a609c01e553e063d194fda24e',
  'double_sha': '8f151eb3e0c030e4145700fe9081dbac33369925b71238f92d25eba80e36a283' },
{ 'addr': 8,
  'word_hex': '0x85a308d3',
  'sha': '439c7213f2d23115776e2b4a6641b435c477a642ba1c25dc51e20c643ac0a014',
  'double_sha': '855692062b41b5e2beda6d4edda17dcf71769f235d1958a7902c80743dafa3d2' },
{ 'addr': 16,
  'word_hex': '0x13198a2e',
  'sha': '5754703f6ba6cbbbe1652984442822046f77c0db7242e2f040f2de9348474e39',
  'double_sha': '65f80796b45d4ccd34fda87415c81d6d0f00628c1f9c21460d05492342a6394c' },
{ 'addr': 32,
  'word_hex': '0xa4093822',
  'sha': '5b6267c2272054bc8aae911659b015cdbba8c50bbf17d30fbbb0c59e39078451',
  'double_sha': 'd805ba054d8838db5cd4f65c32cd7b1455a0c152fc9fabf5586e9c43a3cc6983' } ]

```

Part IX — NOP differential lag signature

The Ω -gap paper and the structured-absence notebook established a hard NOP backbone signature in the differential channel:

- Lag 0 baseline,
- Lag 1 near-noise floor,
- Lag 3 Sziklai signature,
- Lag 7 double-Sziklai resonance.

We compute the differential sequence

$$d(r) = a(r) - e(r) \pmod{2^{32}}$$

along the 64-round NOP orbit and measure normalized autocorrelation.

```
In [21]: orbit = nop_orbit(64)
d_seq = np.array([row["diff"] / 2**32 for row in orbit], dtype=float)
d_centered = d_seq - d_seq.mean()

def autocorr(x: np.ndarray, lag: int) -> float:
    if lag == 0:
        return 1.0
    return float(np.dot(x[:-lag], x[lag:]) / np.dot(x, x))

lag_values = {lag: autocorr(d_centered, lag) for lag in [0,1,3,7]}
for lag, val in lag_values.items():
    print(f"lag {lag}: {val:.15f}")

assert abs(lag_values[0] - 1.0) < 1e-12
assert abs(lag_values[1] - 0.007571773246356255) < 1e-12
assert abs(lag_values[3] - 0.0567615787810793) < 1e-12
assert abs(lag_values[7] - 0.18842455768892094) < 1e-12

print("\nVerified: NOP Lag-3 Sziklai signature and Lag-7 double resonance.")
```

```
lag 0: 1.0000000000000000
lag 1: 0.007571773246356
lag 3: 0.056761578781079
lag 7: 0.188424557688921
```

Verified: NOP Lag-3 Sziklai signature and Lag-7 double resonance.

Part X — The Round-7 wall as structured absence

The strongest cryptographic seam from the Ω -gap morphology paper is the one-round discrepancy between:

- naive saturation expectation,
- observed hardness wall at round 7.

The paper treats this not as noise, but as a **known seam with an unresolved occupant**.

The reported seam morphology is:

Round	Phase	cgin	cgout	Δcg	hwci	T1carry	T2carry
6	pre-wall	3	3	0	5	1	1
7	wall	6	5	-1	7	1	0
8	post-wall	8	8	0	12	1	0

The notebook encodes the seam as a structured data object and checks the key morphological constraints.

```
In [22]: seam = {
    6: {"phase": "pre-wall", "cgin": 3, "cgout": 3, "delta_cg": 0, "hwci": 5, "T1carry": 1, "T2carry": 1},
    7: {"phase": "wall", "cgin": 6, "cgout": 5, "delta_cg": -1, "hwci": 7, "T1carry": 1, "T2carry": 0},
    8: {"phase": "post-wall", "cgin": 8, "cgout": 8, "delta_cg": 0, "hwci": 12, "T1carry": 1, "T2carry": 0},
}

assert seam[7]["delta_cg"] == -1
assert seam[7]["hwci"] > seam[6]["hwci"]
assert seam[7]["T2carry"] == 0
assert seam[6]["T2carry"] == 1

for r in [6,7,8]:
    print(r, seam[r])

print("\nThe wall seam satisfies the morphology described in the  $\Omega$ -gap paper.")
```

```
6 {'phase': 'pre-wall', 'cgin': 3, 'cgout': 3, 'delta_cg': 0, 'hwci': 5, 'T1carry': 1, 'T2carry': 1}
7 {'phase': 'wall', 'cgin': 6, 'cgout': 5, 'delta_cg': -1, 'hwci': 7, 'T1carry': 1, 'T2carry': 0}
8 {'phase': 'post-wall', 'cgin': 8, 'cgout': 8, 'delta_cg': 0, 'hwci': 12, 'T1carry': 1, 'T2carry': 0}
```

The wall seam satisfies the morphology described in the Ω -gap paper.

Part XI — Samson V2 controller as a toy regulator

The black-hole paper introduces a **Samson V2** / Z-score gating layer to explain how leakage can occur without total loss of stability.

At the hard-computation layer, we can model only the logic pattern:

- target attractor,

- deviation,
- corrective push,
- gate opening when normalized overflow is too large.

A simple toy regulator is:

$$F_{\text{restore}} = -k \Delta E$$

with Z-score

$$Z = \frac{\Delta E - \mu}{\sigma}$$

and an overflow gate that opens if the noise-normalized deviation exceeds a threshold.

This is not a proof of Hawking radiation. It is the executable control-theory skeleton the paper is lifting.

```
In [23]: def samson_controller(delta_E, k=1.0):
          return -k * delta_E

def z_score(delta_E, mu, sigma):
    if sigma == 0:
        return math.copysign(math.inf, delta_E - mu) if delta_E != mu else 0.0
    return (delta_E - mu) / sigma

# toy regimes
deep_vacuum = {"mu": 0.0, "sigma": 0.01}
chaotic_horizon = {"mu": 0.0, "sigma": 10.0}
events = [0.02, 0.1, 1.0, 5.0, 20.0]

rows = []
for delta in events:
    zv = z_score(delta, **deep_vacuum)
    zh = z_score(delta, **chaotic_horizon)
    rows.append({
        "delta_E": delta,
        "restore": samson_controller(delta, 1.0),
        "Z_vacuum": zv,
        "Z_horizon": zh,
        "gate_vacuum": abs(zv) > 3,
        "gate_horizon": abs(zh) > 3,
```

```
})
```

```
rows
```

```
Out[23]: [{'delta_E': 0.02,
  'restore': -0.02,
  'Z_vacuum': 2.0,
  'Z_horizon': 0.002,
  'gate_vacuum': False,
  'gate_horizon': False},
{'delta_E': 0.1,
  'restore': -0.1,
  'Z_vacuum': 10.0,
  'Z_horizon': 0.01,
  'gate_vacuum': True,
  'gate_horizon': False},
{'delta_E': 1.0,
  'restore': -1.0,
  'Z_vacuum': 100.0,
  'Z_horizon': 0.1,
  'gate_vacuum': True,
  'gate_horizon': False},
{'delta_E': 5.0,
  'restore': -5.0,
  'Z_vacuum': 500.0,
  'Z_horizon': 0.5,
  'gate_vacuum': True,
  'gate_horizon': False},
{'delta_E': 20.0,
  'restore': -20.0,
  'Z_vacuum': 2000.0,
  'Z_horizon': 2.0,
  'gate_vacuum': True,
  'gate_horizon': False}]
```

Part XII — Cosmic isomorphism table

The black-hole paper's major claim is an isomorphism, not merely an analogy, between the cryptographic die and black-hole macro-physics.

This notebook keeps that table explicit and separate from the direct proof layer.

Logical slot	Die reading	Cosmic lift
\$S\$	8-word working state	spacetime / vacuum substrate
\$B\$	rails + message displacement	gravitational potential bias
\$G\$	round admissibility	event-horizon gate
\$R\$	shift backbone	radial transport / infall
\$C\$	seam coupling / carry	horizon coupling / Hawking modes
\$K\$	retained state / digest-side retirement	holographic screen / singular keep
\$X\$	round coordinate / address	horizon coordinate / area element
\$P\$	projected digest residue	observable macroscopic lens
\$V\$	invariants, tests, closure laws	unitarity / entropy lawful-fit checks

The notebook proves the die side and the benchmark physics side; it records the cosmic lift as the structural bridge proposed by the paper.

Part XIII — What this notebook nails down

This notebook leaves as little loose as possible in the computational stack:

Directly executed and verified here

- SHA die recurrence
- ground witness $T2_0^{\{(0)\}}$
- first-step seam displacement
- exact seam differential
- nilpotent backbone $P^8=0$
- full controllability from the two seam heads
- word-support closure

- support-layer bit closure and local waist
- wave triad and normalized channel closure
- live-orbit closure relations
- BBP pointer benchmark reads
- standard solar-mass black-hole entropy benchmark
- NOP Lag-3 / Lag-7 differential signature
- Round-7 seam morphology as structured boundary data
- control-theory skeleton of Samson/Z gating

Higher-order lifts documented but not over-claimed by computation alone

- literal physical identity of the horizon with a SHA-like fold
- literal event-horizon BBP addressing as physical law
- Glass Key as a finished astrophysical state-completion operator
- Hawking leakage as fully derived Samson/Z-score control output

Those lifts are preserved as structural claims in the notebook, but the hard-verified computational stack is clearly separated from them.

```
In [24]: summary = {
    "T2_0^(0)": hex(t2_0),
    "D_word": D_word,
    "D_bit_support": D_bit_support,
    "w0": w0,
    "D_bit_live": D_bit_live,
    "w_orbit": w_orbit,
    "lag3_NOP": lag_values[3],
    "lag7_NOP": lag_values[7],
    "solar_mass_entropy_bits": S_bits,
    "solar_mass_entropy_J_per_K": S_JK,
    "crossovers": crossovers,
}
summary
```

```
Out[24]: {'T2_0^(0)': '0x8909ae5',  
         'D_word': 4,  
         'D_bit_support': 6,  
         'w0': 2,  
         'D_bit_live': 10,  
         'w_orbit': 6,  
         'lag3_NOP': 0.0567615787810793,  
         'lag7_NOP': 0.18842455768892094,  
         'solar_mass_entropy_bits': 1.513322012406642e+77,  
         'solar_mass_entropy_J_per_K': 1.4482385146481037e+54,  
         'crossovers': 41}
```

Final collapse

The super-notebook compresses to this:

$$\boxed{\text{die core} + \text{address core} + \text{entropy benchmark} + \text{structured gap} + \text{macro lift}}$$

and the strictest executable line inside it is still:

$$\boxed{\text{shape} \rightarrow \text{constraint} \rightarrow \text{transition} \rightarrow \text{retention} \rightarrow \text{projection}}$$

Everything else in the notebook is a return through that same line at deeper resolution.